ELK on Kubernetes Lab Guide

An existing Kubernetes cluster with Linux and Bash

Prepared 15 October 2026. This lab deploys Elasticsearch, Logstash and Kibana using Elastic Cloud on Kubernetes (ECK), collects container logs with Filebeat, and verifies ingestion, retention and recovery.

The examples pin **ECK 3.5.0 and Elastic Stack 9.5.5**, the versions shown in the official documentation checked for this guide. Recheck the compatibility matrix, security fixes and image availability before running. These files have been prepared and statically reviewed; they have **not been deployed to your cluster**.

1 Design and prerequisites

Allow approximately 2 to 3 hours after prerequisites are available. You need basic Kubernetes, YAML, Linux and TLS knowledge. Run commands from the extracted ELK-K8s-Lab directory.

```
Application stdout in elk-demo
  -> node log files
  -> Filebeat DaemonSet in logging-agents
  -> mutual TLS, TCP 5044
  -> Logstash with a persistent queue in logging
  -> authenticated HTTPS, TCP 9200
  -> Elasticsearch data stream logs-kubernetes-lab
  -> Kibana over HTTPS, TCP 5601

ECK in elastic-system manages Elasticsearch, Kibana and Logstash.
```

Filebeat reads node logs and adds Kubernetes metadata. Logstash parses the sample JSON and controls routing. Elasticsearch stores and searches events. Kibana presents them. Logstash is useful for transformations and buffering; a simpler design can send collector output directly to Elasticsearch.

This lab collects only the **elk-demo namespace**. It does not configure Kubernetes audit logging, host journals, metrics or Fleet integrations. The explicit field mapping demonstrates a controlled schema; it is not a complete ECS integration.

Component	Lab allocation	Attention
Elasticsearch	3 pods; 4 GiB RAM and 30 GiB PVC each	Three distinct eligible Linux nodes
Logstash	1 pod; 2 GiB RAM; 10 GiB PVC	4 GB persistent queue; single ingestion instance
Kibana	1 pod; 1 GiB request, 2 GiB limit	Single UI instance
Filebeat	One pod per eligible Linux node	128 MiB request, 512 MiB limit per pod
ECK	512 MiB request, 1 GiB limit	Operator capacity depends on managed objects

Reserve at least about 16 GiB of free schedulable RAM on a three-worker lab cluster, plus Kubernetes, existing workloads and recovery headroom. The stack requests 100 GiB of PVC capacity. These are low-volume starting allocations, not throughput guarantees.

> **ATTENTION** - This is a resilient Elasticsearch lab, not a complete production platform. One Logstash and one Kibana instance remain availability bottlenecks. Three virtual nodes on one physical host do not provide host-failure resilience.

The package contains manifests/, api/ request bodies and scripts/ helpers. Keep the Markdown guide for copyable commands; the PDF is the reading version.

Sources: ECK compatibility, ECK download, Elastic release notes.

2 Inspect the installed cluster

Use dedicated nonproduction resources. Confirm the active context before any change:

```
kubectl config current-context
kubectl version
kubectl get nodes -o wide
kubectl get nodes -L topology.kubernetes.io/zone
kubectl get storageclass
kubectl get csidriver
kubectl get pods -A
kubectl get crd elasticsearches.elasticsearch.k8s.elastic.co
helm list -A
```

A NotFound result for the CRD suggests ECK is not installed. If it exists, inspect the operator version and watched namespaces with the platform owner. Reuse a compatible installation. Do not deploy another operator that watches the same resources.

Provide kubectl, Helm 3, Bash, curl supporting --fail-with-body, OpenSSL, base64 and envsubst from gettext on the workstation. Use a kubectl version supported for the cluster.

```
helm version --short
curl --version
openssl version
command -v envsubst
kubectl auth can-i create customresourcedefinitions.apiextensions.k8s.io
kubectl auth can-i create clusterroles.rbac.authorization.k8s.io
kubectl auth can-i create namespaces
```

ECK installation and Filebeat's read-only ClusterRole require cluster-level permissions. Record the Kubernetes version, CNI, CSI driver, StorageClass, worker names and available resources. If Metrics Server is installed, inspect kubectl top nodes. Also inspect kubectl describe node NODE_NAME for allocated requests; low current utilization does not prove enough schedulable capacity.

Choose storage deliberately

```
kubectl get storageclass YOUR_CLASS -o yaml
```

Use a tested CSI block-storage class with filesystem support and predictable latency. Prefer WaitForFirstConsumer for topology-aware binding and allowVolumeExpansion: true. Understand its reclaimPolicy. Do not modify a shared default class just for the lab; provision a suitable class with the storage administrator if needed.

Each Elasticsearch pod gets a separate ReadWriteOnce PVC. Local PVs can strand data when a node fails. Network-attached block volumes can usually reattach, subject to zone and CSI constraints. Logstash persistent queues do not support NFS.

> **ATTENTION** - PVC retention, PV reclaim policy and Elastic snapshots are separate controls. Retaining a PVC does not protect against namespace deletion, storage failure or unwanted data changes.

Confirm cluster DNS, synchronized node clocks, image pulls from docker.elastic.co and the demo registry, CNI NetworkPolicy enforcement and service-mesh restrictions. Review admission rules: Filebeat uses root and hostPath access and may require a narrowly scoped collector exception. Do not weaken admission policy for the whole cluster.

Sources: Storage recommendations, Persistent queues.

3 Prepare worker nodes and ECK

Persist the host setting

On every eligible Elasticsearch worker, have the administrator apply this through the approved node configuration mechanism:

```
# Run on each worker, not on the kubectl workstation.
printf 'vm.max_map_count=1048576\n' |
  sudo tee /etc/sysctl.d/99-elasticsearch.conf
sudo sysctl --system
sysctl vm.max_map_count
```

Expected: 1048576. Current Elastic guidance uses this for Elasticsearch 8.16 and later. Verify after reboot and node replacement. Managed clusters may require a provider-specific mechanism.

Avoid adding privileged init containers solely to bypass platform restrictions. Disabling mmap is a constrained lab workaround with performance implications; this baseline keeps mmap enabled.

Label three distinct, schedulable workers, replacing the example names:

```
kubectl label node WORKER_A WORKER_B WORKER_C elk-data=true
kubectl get nodes -l elk-data=true
```

Strict anti-affinity intentionally leaves a pod Pending if three eligible nodes are unavailable. Fix capacity, labels or taints rather than weakening placement accidentally.

Set variables and render the manifests

```
cp lab.env.example lab.env
nano lab.env
# Set STORAGE_CLASS and review the pinned versions.
source lab.env
bash scripts/render.sh
kubectl apply -f rendered/00-namespaces.yaml
kubectl apply -f rendered/60-network-policy.yaml
```

The renderer substitutes only STACK_VERSION, STORAGE_CLASS and DEMO_IMAGE. Runtime variables such as ${ELK_ES_PASSWORD} and ${NODE_NAME} remain intact.

> **ATTENTION** - Unrestricted envsubst or an unquoted shell heredoc can replace ECK credential variables with empty workstation values. Use the supplied renderer.

Install only if no compatible operator already exists

```
helm repo add elastic https://helm.elastic.co
helm repo update
helm show chart elastic/eck-operator --version "$ECK_VERSION"
helm install elastic-operator elastic/eck-operator \
  --version "$ECK_VERSION" \
  --namespace elastic-system --create-namespace \
  --set resources.requests.cpu=100m \
  --set resources.requests.memory=512Mi \
  --set resources.limits.memory=1Gi \
  --wait --timeout 10m
kubectl -n elastic-system rollout status \
  statefulset/elastic-operator --timeout=300s
kubectl get crd elasticsearches.elasticsearch.k8s.elastic.co \
  kibanas.kibana.k8s.elastic.co logstashes.logstash.k8s.elastic.co
```

Keep the validating webhook enabled. This is a cluster-scoped installation. For shared clusters, agree namespace scoping and administrative ownership with the platform team. Do not overwrite an existing operator's configuration.

Checkpoint: the operator is Ready, the CRDs exist, and there are no repeated RBAC or webhook failures. Upgrade the operator and CRDs using Elastic's documented procedure when necessary.

Sources: Virtual memory, Install ECK with Helm.

4 Deploy Elasticsearch and Kibana

Review rendered/10-elasticsearch.yaml. It defines three nodes with master, data and ingest roles, explicit resources, strict hostname anti-affinity and per-node PVCs. Mixed roles suit this small cluster; larger deployments may separate master, data and ingest workloads.

```
kubectl apply --dry-run=server -f rendered/10-elasticsearch.yaml
kubectl apply -f rendered/10-elasticsearch.yaml
kubectl -n logging get elasticsearch elk
kubectl -n logging get pods,pvc -w
```

Stop the watch with Ctrl+C after pods are Ready and PVCs Bound, then run:

```
kubectl -n logging wait pod \
  -l elasticsearch.k8s.elastic.co/cluster-name=elk \
  --for=condition=Ready --timeout=900s
kubectl -n logging get pods -o wide
kubectl -n logging get pdb
kubectl apply --dry-run=server -f rendered/20-kibana.yaml
kubectl apply -f rendered/20-kibana.yaml
kubectl -n logging wait pod -l kibana.k8s.elastic.co/name=elk \
  --for=condition=Ready --timeout=600s
```

Confirm that Elasticsearch uses three different nodes. ECK manages discovery, transport TLS and default disruption protection. Do not set discovery.type: single-node or manually configure cluster.initial_master_nodes.

Keep Elasticsearch's automatic heap sizing. The 4 GiB container limit also serves filesystem cache and native memory; do not assign all of it to the JVM. CPU limits can cause throttling, so measure before choosing production limits.

> **ATTENTION** - DeleteOnScaledownOnly preserves Elasticsearch PVCs when its custom resource is deleted, but scale-down can still delete PVCs. Deleting the namespace deletes its PVC objects. The PV reclaim policy then controls the underlying storage.

Establish an authenticated local connection

In a separate terminal, leave this running:

```
kubectl -n logging port-forward --address 127.0.0.1 \
  service/elk-es-http 9200:9200
```

In the original terminal:

```
bash scripts/prepare-access.sh
bash scripts/es.sh GET /
bash scripts/es.sh GET '/_cluster/health?pretty'
bash scripts/es.sh GET '/_cat/nodes?v'
```

The helper verifies the ECK certificate and uses the service DNS name through curl --resolve over the tunnel. It stores the initial admin credential in a private .secrets/es.netrc file, avoiding a password in the curl command line.

Checkpoint: the API shows the intended version and three nodes, and health becomes green. A green empty cluster does not prove replicated log ingestion; validate the data stream later.

Use elastic only for bootstrap administration. Never put it into Filebeat, application configuration or shared dashboard accounts. Protect .secrets, kubeconfig and Kubernetes Secret access.

Sources: Deploy Elasticsearch, PVC settings, Compute resources.

5 Install retention and the data stream

Install the index template before starting ingestion. logs-kubernetes-lab uses one primary shard, one replica and a five-second refresh interval. Its ILM policy rolls over at a 10 GB primary shard or one day and deletes rolled-over indices after seven days.

```
bash scripts/es.sh PUT /_ilm/policy/elk-lab-7d \
  api/ilm-policy.json
bash scripts/es.sh PUT /_index_template/elk-lab-logs \
  api/index-template.json
bash scripts/es.sh POST \
  /_index_template/_simulate_index/logs-kubernetes-lab
bash scripts/es.sh PUT /_data_stream/logs-kubernetes-lab
bash scripts/es.sh GET /_data_stream/logs-kubernetes-lab
```

The template simulation should show the intended mapping and settings. Its high priority applies only to the exact lab stream, not all logs-* integrations. Creating the data stream is a one-time step; if it exists, inspect it instead of deleting data to repeat the command.

dynamic: false limits indexing to the fields explicitly mapped for the lab. Other fields remain in _source but are not searchable or aggregatable. Extend the schema deliberately. Use approved ECS/integration templates when adopting Elastic integrations.

> **ATTENTION** - The seven-day delete age is measured from rollover for a rolled-over backing index, not from each event's timestamp. Retention includes rollover and processing delay; it is not an exact per-record deletion guarantee.

Logstash sets manage_template: false and ilm_enabled: false because an administrator creates these objects. Elasticsearch still executes the ILM policy attached by the template. Plugin-side ILM management and server-side lifecycle execution are different controls.

Estimate storage from measured indexed data:

```
Raw Elasticsearch storage
  = daily primary-index growth x retention days
    x (1 + replica count) / target utilization
```

Example: 10 GiB/day of primary growth, seven days, one replica and 70% target utilization gives about 200 GiB, before extra allowance for rollover timing, system indices, growth and recovery. Raw application log volume can differ substantially from indexed size.

The lab's 90 GiB Elasticsearch capacity is deliberately small. Keep the generator at low volume. Alert before disk watermarks prevent allocation or block writes; ILM cannot instantly repair severe undersizing.

Checkpoint: GET /logs-kubernetes-lab/_ilm/explain shows managed backing indices. The primary and replica should be allocated to different Elasticsearch nodes.

Sources: ILM with data streams, Elasticsearch output plugin.

6 Secure and deploy Logstash

ECK supplies TLS and credentials for the Elasticsearch association. Secure the Filebeat-to-Logstash listener separately.

The certificate helper creates a dedicated lab CA, a server certificate with the Service DNS names and a client certificate. It uploads the required leaf keys and certificates. The CA private key stays local.

```
bash scripts/create-lab-certs.sh
kubectl -n logging get secret logstash-ingest-tls
kubectl -n logging-agents get secret filebeat-client-tls
kubectl apply --dry-run=server -f rendered/30-logstash.yaml
kubectl apply -f rendered/30-logstash.yaml
kubectl -n logging get logstash ingest
kubectl -n logging wait pod -l logstash.k8s.elastic.co/name=ingest \
  --for=condition=Ready --timeout=600s
kubectl -n logging get service ingest-ls-beats
kubectl -n logging logs ingest-ls-0 --tail=100
```

Checkpoint: ClusterIP service on 5044, pipeline started, certificates loaded and no repeated authentication failures.

The listener requires a certificate signed by the ingestion CA. Its private key is converted to PEM PKCS8, a requirement of the Beats input. The output uses ECK's ELK_ES_* variables with full certificate verification.

> **ATTENTION** - Mutual TLS proves possession of a certificate from a trusted CA; it is not per-application authorization. This lab shares one collector identity. Production needs a certificate issuance, rotation and tenant-access design.

The same server certificate also covers the internal Logstash API service on port 9600. ECK configures HTTPS for this API and probes its pod address; do not bind it only to loopback. The baseline NetworkPolicy, applied before the workloads, blocks ordinary pod access to 9600. Limit administrative port-forward rights and add authenticated API access if monitoring needs it.

The leaf certificates expire in 90 days. Monitor expiry and renew in advance. The script refuses to overwrite an existing CA. Rotate trust with overlap; never replace a live CA blindly. After Secret changes, verify reload behaviour and perform controlled restarts if needed.

Durable queue and memory

The persistent queue and DLQ live beneath /usr/share/logstash/data on a 10 GiB PVC. The 4 GB queue and 1 GB DLQ leave room for metadata and overhead. queue.checkpoint.writes: 1 prioritizes durability and can substantially reduce throughput. Benchmark before changing it.

A PQ is local to a Logstash instance and is not replicated. Retries can duplicate events; this is not exactly-once delivery. Use suitable event IDs and deduplication when required. A DLQ captures certain failed Elasticsearch documents, not all transport failures. Monitor it and plan reprocessing.

The 2 GiB container uses a 768 MiB heap and 384 MiB direct-memory cap, leaving native-memory headroom. Watch OOM kills, throttling and queue growth.

ECK's generated Logstash identity avoids the elastic superuser but is broader than a tightly scoped writer. For strict production least privilege, pre-create lifecycle objects and use a separate credential limited to the required stream and actions, stored through approved Secrets or a secret manager.

Sources: Logstash ECK configuration, Mutual TLS for Filebeat, Logstash API security.

7 Collect and verify logs

Review rendered/40-filebeat.yaml. It mounts /var/log read-only and stores the registry at /var/lib/elk-lab-filebeat on each node. It does not mount the Docker socket. RBAC grants only get/list/watch on pods, nodes and namespaces.

```
kubectl apply --dry-run=server -f rendered/40-filebeat.yaml
kubectl apply -f rendered/40-filebeat.yaml
kubectl -n logging-agents rollout status daemonset/filebeat \
  --timeout=300s
kubectl -n logging-agents get pods -o wide
kubectl apply -f rendered/50-demo.yaml
kubectl -n elk-demo rollout status deployment/log-generator \
  --timeout=180s
kubectl -n elk-demo logs deployment/log-generator --tail=10
```

The generator emits ELK_LAB_OK and an increasing sequence number every two seconds. Filebeat filestream follows Kubernetes log symlinks and parses the container envelope. Logstash places decoded JSON beneath app, keeping application keys separate from Kubernetes metadata.

After about one to two minutes:

```
bash scripts/es.sh GET '/logs-kubernetes-lab/_count?pretty'
bash scripts/es.sh GET \
  '/logs-kubernetes-lab/_search?q=app.message:ELK_LAB_OK&size=3&pretty'
bash scripts/es.sh GET '/logs-kubernetes-lab/_ilm/explain?pretty'
kubectl -n logging-agents logs daemonset/filebeat --tail=80
```

Checkpoint: count increases; app.message is ELK_LAB_OK; app.sequence is numeric; namespace is elk-demo; pod and node names are present. Investigate missing metadata even if raw events arrive.

> **ATTENTION** - Filebeat 9 fingerprinting can wait for enough file bytes before harvesting. The continuous generator crosses the default threshold; one short test line may not appear immediately.

This manifest assumes normal Linux containerd/CRI-O log paths. Inspect symlinks and targets on your nodes. Older Docker or custom layouts may need additional read-only mounts. SELinux/OpenShift may require a collector-specific policy. Do not grant broad privileges as the first fix.

The DaemonSet intentionally has no blanket tolerations. Place the demo on an eligible worker or add only required tolerations. Registry persistence reduces rereads after restarts, but node loss can still lose unshipped local logs.

The glob collects active symlinks for elk-demo. Test rotation, compression and long outages before expanding scope. Evaluate the official rotated /var/log/pods collection recipe for your version. Avoid collecting both paths without a duplication plan.

A ConfigMap subPath mount does not update in place. Restart the DaemonSet deliberately after changing its configuration. The lab template is installed through the API; do not rely on filebeat setup through a Logstash output.

Sources: Filebeat on Kubernetes, Metadata enrichment.

8 Use Kibana and apply network restrictions

In a separate terminal:

```
kubectl -n logging port-forward --address 127.0.0.1 \
  service/elk-kb-http 5601:5601
```

Open https://localhost:5601. The lab certificate includes localhost and 127.0.0.1. Retrieve its public certificate, inspect its fingerprint and trust it using the workstation/browser's approved trust mechanism:

```
kubectl -n logging get secret elk-kb-http-certs-public \
  -o jsonpath='{.data.ca\.crt}' |
  base64 --decode > .secrets/kibana-ca.crt
openssl x509 -in .secrets/kibana-ca.crt -noout \
  -subject -issuer -dates -fingerprint -sha256
```

Use an organizationally trusted certificate for shared access. Do not globally disable certificate verification.

Retrieve the bootstrap password locally; avoid shared terminals and screenshots:

```
kubectl -n logging get secret elk-es-elastic-user \
  -o jsonpath='{.data.elastic}' | base64 --decode
printf '\n'
```

Sign in as elastic for initial setup. Create a data view named logs-kubernetes-lab with @timestamp as its time field. In Discover, choose the last 15 minutes and filter:

```
kubernetes.namespace : "elk-demo" and app.message : "ELK_LAB_OK"
```

Add pod name, node name, app.sequence and app.level columns. Save the search. Create named viewer accounts with read and view_index_metadata only on the intended stream and read-only Discover/Dashboard access in the intended space.

Apply the baseline policy

```
kubectl apply -f rendered/60-network-policy.yaml
kubectl -n logging get networkpolicy
```

This limits ingress to logging pods. It allows Elasticsearch peer traffic, Logstash/Kibana access to Elasticsearch, ECK from elastic-system and the collector connection to Logstash. Narrow the operator-namespace allowance to verified pod labels when appropriate.

The policy does not restrict egress or install an Ingress controller. Verify actual CNI enforcement and recheck ingestion. Port-forward is an administrative path, not evidence that ordinary pod-to-pod NetworkPolicy enforcement works.

For production default-deny egress, explicitly permit and test these flows:

Source	Destination	Flow
Filebeat	Logstash	TCP 5044 with mutual TLS
Filebeat and ECK	Kubernetes API	Cluster API endpoint and port
Components	DNS	UDP/TCP 53; account for NodeLocal DNS
Logstash and Kibana	Elasticsearch	TCP 9200 HTTPS
Elasticsearch peers	Elasticsearch peers	TCP 9300
ECK	Managed APIs	Required application ports
Approved ingress controller	Kibana	TCP 5601 with verified upstream HTTPS
Elasticsearch	Snapshot store	Provider HTTPS
Monitoring collectors	Monitored APIs	Only selected monitoring flows

For shared Kibana access, use an approved private Ingress/Gateway, trusted certificates and authentication. Match upstream HTTPS and CA validation. Keep Elasticsearch and Logstash internal unless a separate access design requires otherwise.

Source: Kubernetes NetworkPolicy.

9 Test recovery and monitor health

Perform disruptions only with disposable lab data. Record a healthy baseline:

```
mkdir -p evidence
kubectl -n logging get pods,pvc,pdb -o wide > evidence/before.txt
bash scripts/es.sh GET '/_cluster/health?pretty' > evidence/health.json
bash scripts/es.sh GET /_data_stream/logs-kubernetes-lab
bash scripts/es.sh GET '/_cat/shards?v'
```

Identify the stream's backing indices in the data-stream response and confirm their primary/replica allocation.

Restart one Elasticsearch pod

With three healthy nodes and allocated replicas:

```
kubectl -n logging delete pod elk-es-main-0
kubectl -n logging get pods -w
```

Do not delete its PVC. After recreation, verify the same PVC, older searchable events, continuing new events and green health. Health may briefly turn yellow. Wait for green before another disruption.

> **ATTENTION** - Direct pod deletion bypasses eviction API PDB protection. PDBs help with voluntary evictions, not simultaneous failures or arbitrary deletion. Do not force-drain when the budget blocks maintenance.

Three master-eligible nodes can retain a majority after one fails. Data availability also depends on replicas, placement and storage. Hostname separation is not zone-level shard allocation awareness.

Restart Logstash and Filebeat

```
kubectl -n logging delete pod ingest-ls-0
kubectl -n logging wait pod ingest-ls-0 \
  --for=condition=Ready --timeout=600s
kubectl -n logging-agents rollout restart daemonset/filebeat
kubectl -n logging-agents rollout status daemonset/filebeat \
  --timeout=300s
```

Verify resumed ingestion, unchanged PVC binding and no repeated delivery failures. This smoke test does not prove zero loss or duplication. A full durability test uses known event IDs, an induced downstream outage, queue measurements and reconciliation after recovery.

Monitor:

- Elasticsearch health, shards, disks, indexing rejections, JVM pressure and latency.
- Logstash queue growth, event rates, output failures, DLQ, memory and restarts.
- Filebeat Ready count, retries, harvesting errors, dropped events and node disk pressure.
- ECK errors, PVC conditions, certificate expiry and pending workloads.
- End-to-end arrival delay for an expected application event.

Deliver alerts outside the logging stack's own failure domain where practical. Set thresholds from workload baselines. Investigate disk growth around 70% and rising queue occupancy before saturation.

Sources: Disruption budgets, Persistent queues.

10 Back up and prove a restore

Replicas support availability. Use Elastic snapshots for recovery, stored outside the cluster's failure domain. ECK does not automatically provision your snapshot repository. Copying Elasticsearch data directories or relying only on volume snapshots is not a supported substitute.

Choose a supported repository and configure identity using the official ECK provider recipe. Prefer workload identity where available; generic clusters may need ECK secureSettings. Never embed cloud keys in these manifests.

The following exercise assumes a repository named lab-repository. For S3, after the bucket, permissions and Elasticsearch identity exist, a registration body could be:

```
{
  "type": "s3",
  "settings": {
    "bucket": "REPLACE_WITH_BACKUP_BUCKET",
    "base_path": "elk-lab"
  }
}
```

Save the correct provider-specific body as api/repository.json. The placeholder is not a usable bucket.

```
bash scripts/es.sh PUT /_snapshot/lab-repository \
  api/repository.json
bash scripts/es.sh POST /_snapshot/lab-repository/_verify
```

Require successful node verification. For S3-compatible stores, check endpoint, CA trust and repository compatibility.

Non-destructive restore exercise

Snapshot a probe index and restore it under a different name:

```
bash scripts/es.sh PUT '/elk-backup-probe/_doc/1?refresh=true' \
  api/probe-document.json
bash scripts/es.sh PUT \
  '/_snapshot/lab-repository/elk-lab-probe?wait_for_completion=true' \
  api/restore-probe.json
bash scripts/es.sh GET /_snapshot/lab-repository/elk-lab-probe
bash scripts/es.sh POST \
  /_snapshot/lab-repository/elk-lab-probe/_restore \
  api/restore-renamed.json
bash scripts/es.sh GET \
  '/_cluster/health/elk-backup-probe-restored?wait_for_status=green&timeout=120s'
bash scripts/es.sh GET '/elk-backup-probe-restored/_doc/1'
```

For repeated exercises, choose a new snapshot and restored-index name. Require snapshot state SUCCESS, no failed shards, and restored message ELK_SNAPSHOT_RESTORE_OK. Record elapsed restore time. This exercise excludes global state and feature states.

The probe proves basic repository and restore operation. Before production, restore actual data streams and required feature states into a separate cluster; verify Kibana objects, permissions and recovery objectives.

Set a Snapshot Lifecycle Management schedule and retention aligned with the recovery point objective. Alert on failure. Back up manifests and secret recovery procedures separately. Allow only one writer cluster per shared repository; register other clusters read-only where appropriate.

> **ATTENTION** - A configured backup job is not proof of recovery. Completion requires a successful snapshot and a verified restore.

Sources: ECK snapshot repositories, Snapshot and restore.

11 Troubleshoot by symptom

Symptom	First checks	Corrective direction
Pod Pending	Labels, taints, requests, anti-affinity	Supply three eligible nodes and capacity
PVC Pending	Class, CSI events, zones	Fix provisioning/topology; delayed binding can be expected
Bootstrap failure	Logs and vm.max_map_count	Persist the host setting on every eligible node
OOMKilled	Limits and JVM/native memory	Size from measurements; preserve non-heap headroom
Logstash startup failure	Pipeline, Secret paths, PKCS8 key	Preserve runtime variables and correct key format
TLS errors	SAN, CA, expiry, clock	Correct identity/trust; keep verification enabled
401 or 403	ECK association and output target	Correct credential scope; avoid superuser substitution
No events	File path, metadata, queue, time filter	Trace each hop; allow fingerprint and refresh delays
Fields not searchable	Mapping and JSON target	Add intentional mappings; inspect _source
Yellow/red health	Allocation explanation and disks	Recover replicas/capacity; do not hide failures
Read-only index	Watermark and ILM state	Resolve disk pressure, then verify block recovery
Failure after policy	CNI, labels, DNS/API	Repair the exact allowlist
Duplicate logs	Registry, retries, input paths	Preserve state; design deduplication where required

```
kubectl -n logging describe pod POD_NAME
kubectl -n logging describe pvc PVC_NAME
kubectl -n logging get events --sort-by=.lastTimestamp
kubectl -n logging logs POD_NAME --all-containers --tail=150
kubectl -n logging logs POD_NAME -c CONTAINER_NAME --previous
kubectl -n elastic-system logs statefulset/elastic-operator --tail=100
bash scripts/es.sh GET '/_cat/allocation?v'
bash scripts/es.sh GET '/_cluster/allocation/explain?pretty'
bash scripts/es.sh GET '/logs-kubernetes-lab/_ilm/explain?pretty'
```

Allocation explain without a body selects an unassigned shard if one exists. A response saying none exist is not itself a fault.

For upgrades, keep versioned manifests, review release notes, verify the compatibility matrix, test snapshots and follow the supported order. Elasticsearch in-place downgrade is not a rollback plan. Recover using supported snapshot/version procedures. Change ECK custom resources rather than editing the StatefulSets it owns.

12 Completion and production readiness

The lab is complete when:

- Three Elasticsearch pods occupy separate nodes, PVCs are Bound and health is green.
- Both ingestion hops use certificate verification; Filebeat presents a client certificate.
- Sample events contain searchable application, pod, node and namespace fields.
- The data stream has a replica and the intended ILM policy.
- Kibana Discover shows events over HTTPS.
- Restart checks recover ingestion without PVC deletion.
- Network restrictions are tested with the actual CNI.
- Snapshot/restore succeeds, or is explicitly recorded as pending external repository setup.

Area	Production evidence needed
Capacity	Measured ingest, retention, queries, recovery headroom and cost
Availability	Failure-domain placement and multiple Logstash/Kibana replicas
Storage	CSI performance, expansion, reclaim policy and failure recovery
Access	Named users, scoped roles, encrypted Secrets and credential rotation
Network	Tested ingress/egress allowlists and trusted shared access
Data quality	Schema, multiline parsing, timestamps, redaction and tenant routing
Durability	Queue sizing, log-rotation testing and verified restores
Operations	Alert owners, renewals, upgrades and incident runbooks

The output routes all accepted logs to one lab stream. This is not tenant isolation. Redact secrets and sensitive data close to the source. Prevent uncontrolled labels or JSON fields from producing mapping growth.

Run multiple Logstash replicas on different nodes for production availability. Test long-lived Beats connection distribution: one Service plus loadbalance enabled does not prove equal backend usage. Preserve and drain each replica's queue before removing it.

13 Clean up with a data-retention decision

Stop the generator first:

```
kubectl -n elk-demo scale deployment/log-generator --replicas=0
# In a separate terminal, keep this tunnel running:
kubectl -n logging port-forward --address 127.0.0.1 \
  service/ingest-ls-api 9600:9600
# In the original terminal:
curl --noproxy '*' --fail --cacert .secrets/ca.crt \
  --resolve ingest-ls-api.logging.svc:9600:127.0.0.1 \
  https://ingest-ls-api.logging.svc:9600/_node/stats/pipelines
```

Wait for the pipeline queue events_count to reach zero. The tunnel binds locally and verifies the API certificate; NetworkPolicy protects the in-cluster endpoint. Export evidence and complete required snapshots.

For a disposable lab, remove components in this order:

```
kubectl delete -f rendered/50-demo.yaml
kubectl delete -f rendered/40-filebeat.yaml
# DESTRUCTIVE TO SERVICES: only after queue and backup checks.
kubectl delete -f rendered/20-kibana.yaml
kubectl delete -f rendered/30-logstash.yaml
kubectl delete -f rendered/10-elasticsearch.yaml
kubectl -n logging get pvc
kubectl get pv
```

Do not delete logging if you intend to keep its PVCs. Review remaining PVCs individually and their PV reclaim policies. Check Logstash retention independently of Elasticsearch's PVC policy.

Keep shared ECK resources. Never delete cluster-wide ECK CRDs as routine cleanup. Remove only the node labels you added, revoke lab identities, close port-forwards and remove local secrets using your approved process.

The node directory /var/lib/elk-lab-filebeat remains after DaemonSet deletion. Remove it only after confirming no collector uses it and replay evidence is no longer needed.

References and version review

The linked official documentation governs version-specific behaviour. Recheck compatibility, plugin options, licensing/support requirements and provider-specific settings before production.

- ECK Logstash quickstart
- ECK 3.5 API reference
- Upgrade ECK
- Restore a snapshot